# 1. Software Requirements Specification

**HYBRID_APP_ADA**

| Document Control | |
|---|---|
| Author(s) | Michael Gardner |
| Version | 2.1.0 |
| Status | Released |
| Status Date | 2025-12-14 |
| SPDX-License-Identifier | BSD-3-Clause |
| License File | See the LICENSE file in the project root |
| Copyright | © 2025 Michael Gardner, A Bit of Help, Inc. |

| Project Profile | |
|---|---|
| Variant | application |
| Library Role | N/A |
| Application Role | cli |
| Assurance | spark-targeted |
| Execution | sequential |
| Parallelism | none |
| Platform | desktop, server, embedded |
| Execution Environment | linux, macos, windows, ada-runtime |
| Processor Architecture | amd64, arm64, arm32 |
| Deployment | native |

# Table of Contents

# 2. Introduction

## 2.1. Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **Hybrid_App_Ada**, a professional Ada 2022 application starter demonstrating hybrid DDD/Clean/Hexagonal architecture with functional error handling.

## 2.2. Scope

**Hybrid_App_Ada** provides:

- A professional 5-layer application starter based on hexagonal architecture.
- Static dependency injection via Ada generics.
- Railway-oriented programming with Result monads.
- Automated architecture boundary enforcement.
- A comprehensive built-in test structure and build automation.
- Educational documentation and examples for teams adopting the architecture.
- Production-ready code quality standards.

## 2.3. Definitions and Acronyms

| Term | Definition |
|---|---|
| CLI | Command Line Interface. |
| DDD | Domain-Driven Design — strategic and tactical patterns for complex software. |
| DI | Dependency Injection. |
| DTO | Data Transfer Object. |
| Hexagonal Architecture | Ports & Adapters pattern isolating business logic from infrastructure. |
| Result Monad | Functional pattern for error handling without exceptions. |
| SPARK | Ada subset for formal verification. |

## 2.4. References

- Ada 2022 Reference Manual (ISO/IEC 8652:2023).
- SPARK 2014 Reference Manual.
- **Clean Architecture** (Robert C. Martin).
- **Domain-Driven Design** (Eric Evans).
- Railway-Oriented Programming (Scott Wlaschin).
- Hexagonal Architecture (Alistair Cockburn).
- `functional` crate v3.0.0 documentation.

# 3. Overall Description

## 3.1. Product Perspective

**Hybrid_App_Ada** is a standalone application starter implementing hexagonal architecture with clean separation between domain logic, application use cases, infrastructure adapters, presentation adapters, and bootstrap wiring.

This application profile uses the same hybrid Domain/Application/Infrastructure core as **Hybrid_Lib_Ada**, but expresses the outer boundary through `Presentation` and `Bootstrap` rather than the library's `API`, `API.Desktop`, and `API.Operations` pattern.

| |
|---|
| **Bootstrap**<br>Composition root wires generic use cases, ports, and adapters. |
| ↓ |
| **Presentation**<br>CLI adapter parses input, invokes Application ports, and maps errors to user-facing messages. |
| ↓ |
| **Application → Infrastructure → Domain**<br>Use cases orchestrate workflow; adapters implement outbound ports; Domain remains pure. |

Supporting architecture guidance is maintained in `docs/guides/`. Supporting UML diagrams are maintained in `docs/diagrams/`.

| Layer | Purpose |
|---|---|
| Domain | Pure business logic, value objects, and error types with zero external dependencies. |
| Application | Use cases, commands, and inbound/outbound ports. |
| Infrastructure | Driven adapters implementing outbound ports. |
| Presentation | Driving adapters such as CLI or API front ends. |
| Bootstrap | Composition root wiring all dependencies. |

## 3.2. Product Features

1. **Hexagonal Architecture**: 5-layer clean architecture.
2. **Static Dependency Injection**: Compile-time wiring via generics.
3. **Railway-Oriented Programming**: Result monad error handling.
4. **Architecture Enforcement**: Automated boundary validation.
5. **Test Infrastructure**: Custom test framework and structured test organization.
6. **Build Automation**: Make targets for common developer workflows.
7. **Formal Verification**: SPARK-compatible domain and largely SPARK-friendly application logic.
8. **Result Combinators**: `Bimap`, `Ensure`, `With_Context`, `Fallback`, `Recover`, and `Tap` support in v2.0.0.

## 3.3. User Classes

| User Class | Description |
|---|---|
| Ada Developers | Developers starting new applications with modern best practices. |
| Application Developers | Developers learning or adopting hexagonal architecture patterns in Ada. |
| Educators | Instructors teaching clean architecture principles. |
| Team Leads | Developers establishing architectural standards for new projects. |

## 3.4. Operating Environment

| Requirement | Specification |
|---|---|
| Platforms | POSIX (Linux, macOS, BSD), Windows 11, Embedded. |

| Requirement | Specification |
|---|---|
| Ada Compiler | GNAT FSF 13+ or GNAT Pro. |
| Ada Version | Ada 2022. |
| Build System | Alire 2.0+, GNU Make. |
| Dependencies | `functional ^3.0.0` |

### 3.5. Constraints

| Constraint | Rationale |
|---|---|
| Ada 2022 | Required for modern language features and contract-based design. |
| Architecture Enforcement | Boundary rules must remain mechanically verifiable. |
| GNAT 13+ | Required compiler baseline. |
| Static DI | Compile-time wiring and zero runtime DI overhead. |

## 4. Interface Requirements

### 4.1. User Interfaces

The starter provides a CLI-driven presentation adapter. Additional driving adapters may be added by derivative projects without changing the architectural rules established by this SRS.

### 4.2. Software Interfaces

#### 4.2.1. Build and Package Interfaces

```
[[depends-on]]
functional = "^3.0.0"
```

#### 4.2.2. Application Entry Flow

```
with Bootstrap.CLI;

procedure Main is
begin
   Bootstrap.CLI.Run;
end Main;
```

#### 4.2.3. Inbound and Outbound Ports

- Inbound ports represent application use cases.
- Outbound ports represent infrastructure capabilities exposed to the application layer.
- Ports use static signatures and compile-time wiring rather than runtime interface dispatch.

### 4.3. Communications Interfaces

None are required by the starter itself. Derived projects may add network-facing adapters while preserving the application and domain boundaries.

### 4.4. Hardware Interfaces

The starter does not require direct hardware integration, but it must permit embedded-target adapters when a derived project needs them.

# 5. Functional Requirements

## 5.1. Domain Layer (FR-01)

**Priority:** Critical **Description:** Pure business logic with zero external dependencies.

| ID | Requirement |
|---|---|
| FR-01.1 | Value objects shall be immutable after successful creation. |
| FR-01.2 | Validation shall occur during value object creation. |
| FR-01.3 | Domain code shall have zero infrastructure dependencies. |
| FR-01.4 | Business rules shall be implemented as pure functions with no side effects. |
| FR-01.5 | Result monads shall handle expected errors instead of exceptions. |
| FR-01.6 | Domain shall provide raw data rather than formatted presentation output. |

**Acceptance Criteria (FR-01):**

- Value creation performs validation before producing a valid object.
- Domain packages remain free of I/O and infrastructure dependencies.
- Expected failures are represented as Result errors rather than propagated exceptions.

## 5.2. Application Layer (FR-02)

**Priority:** Critical **Description:** Use case orchestration, port definitions, and output formatting.

| ID | Requirement |
|---|---|
| FR-02.1 | The application layer shall define inbound ports representing use case interfaces. |
| FR-02.2 | The application layer shall define outbound ports representing infrastructure dependencies. |
| FR-02.3 | Use cases shall orchestrate domain logic and outbound port calls. |
| FR-02.4 | Commands shall be DTOs with bounds larger than Domain validation bounds where applicable. |
| FR-02.5 | The application layer shall re-export `Domain.Error` for Presentation access. |
| FR-02.6 | Output formatting shall occur in the application layer rather than in the domain. |

**Acceptance Criteria (FR-02):**

- Inbound and outbound port contracts are defined in the application layer.
- Use cases coordinate domain behavior and adapter interactions without introducing presentation concerns.
- Formatting logic remains outside the domain.

## 5.3. Infrastructure Layer (FR-03)

**Priority:** High **Description:** Concrete adapter implementations.

| ID | Requirement |
|---|---|
| FR-03.1 | Infrastructure shall implement outbound port interfaces defined by the application layer. |
| FR-03.2 | Infrastructure shall adapt external systems to domain and application types. |
| FR-03.3 | Infrastructure exceptions shall be handled at architectural boundaries. |
| FR-03.4 | Boundary exception handling shall convert failures into Result errors. |
| FR-03.5 | The starter shall provide a console writer adapter. |

**Acceptance Criteria (FR-03):**

- Outbound port implementations are concrete adapters owned by Infrastructure.
- Adapter failures are converted to Result errors before crossing boundaries.
- The starter includes a working console writer adapter.

## 5.4. Presentation Layer (FR-04)

**Priority:** High **Description:** User-interface adapters for the CLI starter.

| ID | Requirement |
|---|---|
| FR-04.1 | Presentation shall not access Domain directly. |
| FR-04.2 | Presentation shall use Application-owned error types for user-facing error handling. |
| FR-04.3 | Presentation shall use Application command types for input. |
| FR-04.4 | The CLI adapter shall parse command-line arguments. |
| FR-04.5 | The CLI adapter shall provide user-friendly error messages. |
| FR-04.6 | The CLI adapter shall map exit codes as 0 for success and 1 for error. |

**Acceptance Criteria (FR-04):**

- Presentation accesses the application layer but not the domain directly.
- CLI argument parsing produces application commands.
- Failures result in user-facing messages and the documented exit code mapping.

## 5.5. Bootstrap Layer (FR-05)

**Priority:** High **Description:** Composition root with dependency wiring.

| ID | Requirement |
|---|---|
| FR-05.1 | Bootstrap shall wire generic instantiations at compile time. |
| FR-05.2 | Bootstrap shall connect ports to adapters. |
| FR-05.3 | The main procedure shall remain minimal and delegate to Bootstrap. |
| FR-05.4 | The starter shall use a single-project structure rather than aggregate projects. |
| FR-05.5 | Dependency wiring shall use static compile-time resolution. |

**Acceptance Criteria (FR-05):**

- Bootstrap acts as the composition root.
- Main delegates to bootstrap wiring rather than carrying application logic.
- Wiring remains static and type-safe.

## 5.6. Error Handling (FR-06)

**Priority:** Critical **Description:** Railway-oriented programming with Result monad.

| ID | Requirement |
|---|---|
| FR-06.1 | No exceptions shall cross architectural layer boundaries. |
| FR-06.2 | Result monads shall represent all fallible operations. |
| FR-06.3 | Error types shall contain kind and message information. |
| FR-06.4 | Error kinds shall include `Validation_Error`, `Parse_Error`, `Not_Found_Error`, `IO_Error`, and `Internal_Error`. |
| FR-06.5 | The Result API shall provide `Is_Ok` and `Is_Error` predicates. |

| ID | Requirement |
|---|---|
| FR-06.6 | The Result API shall provide accessors for success values and error information. |
| FR-06.7 | The Result API shall support `And_Then_Into` for cross-type chaining. |
| FR-06.8 | The Result API shall support `Bimap` for transforming success and error values. |
| FR-06.9 | The Result API shall support `Ensure` for postcondition-style validation. |
| FR-06.10 | The Result API shall support `With_Context` for error context enrichment. |
| FR-06.11 | The Result API shall support `Fallback` for default values on error. |
| FR-06.12 | The Result API shall support `Recover` for converting errors into success values. |
| FR-06.13 | The Result API shall support `Tap` for side effects without changing the Result. |

**Acceptance Criteria (FR-06):**

- Expected failures are represented as Results.
- Result combinators are available for application-level workflow composition.
- No exceptions are used as the normal cross-boundary error protocol.

**Error Handling Policy Note:** Expected failures shall be represented as `Result` errors rather than propagated exceptions. Unexpected runtime failures (for example `Program_Error` or `Storage_Error`) remain outside this policy and shall be contained at architectural boundaries using `Functional.Try` or equivalent boundary handling.

## 5.7. Dependency Injection (FR-07)

**Priority:** Critical **Description:** Static dependency injection via Ada generics.

| ID | Requirement |
|---|---|
| FR-07.1 | Use cases shall be implemented using generic packages or generic functions where appropriate. |
| FR-07.2 | Ports shall be passed as generic parameters or equivalent static contracts. |
| FR-07.3 | Compile-time instantiation shall occur in Bootstrap. |
| FR-07.4 | The DI approach shall introduce no runtime dispatch overhead. |
| FR-07.5 | The DI approach shall remain type-safe. |
| FR-07.6 | The DI mechanism shall not require heap allocation. |

**Acceptance Criteria (FR-07):**

- Compile-time wiring replaces runtime service-location patterns.
- Type errors in wiring are detected by the compiler.
- DI introduces no runtime heap dependence.

## 5.8. Architecture Validation (FR-08)

**Priority:** High **Description:** Automated boundary enforcement.

| ID | Requirement |
|---|---|
| FR-08.1 | Automated checks shall validate that Presentation cannot access Domain directly. |
| FR-08.2 | Automated checks shall validate that Infrastructure can access Domain. |
| FR-08.3 | Automated checks shall validate that Application accesses Domain only. |
| FR-08.4 | Automated checks shall validate that Domain has zero dependencies. |

| ID | Requirement |
|---|---|
| FR-08.5 | Architecture validation shall be implemented via `scripts/arch_guard.py` for the selected hybrid architecture profile. |
| FR-08.6 | The validation script shall be run during normal developer build workflow and in CI/CD, including through a build target such as `make check-arch`. |

**Acceptance Criteria (FR-08):**

- Architecture validation can be run automatically during local development and in CI/CD.
- Violations of layer boundaries are reported mechanically rather than only by convention.

# 6. Quality and Cross-Cutting Requirements

## 6.1. Performance (NFR-01)

| ID | Requirement |
|---|---|
| NFR-01.1 | Static dispatch overhead shall be zero at runtime because wiring occurs at compile time. |
| NFR-01.2 | Heap allocation shall be avoided in hot paths. |
| NFR-01.3 | Result monad overhead shall remain minimal and stack-based. |
| NFR-01.4 | A clean build should complete in under 30 seconds on a typical development machine. |
| NFR-01.5 | The full test suite should execute in under 5 seconds on a typical development machine. |

## 6.2. Reliability (NFR-02)

| ID | Requirement |
|---|---|
| NFR-02.1 | All automated tests shall pass. |
| NFR-02.2 | The application shall not introduce memory leaks in supported environments. |
| NFR-02.3 | Error handling for expected failures shall be deterministic and exception-free across boundaries. |
| NFR-02.4 | Architectural boundaries shall remain type-safe and compiler-verifiable. |

## 6.3. Portability (NFR-03)

| ID | Requirement |
|---|---|
| NFR-03.1 | The starter shall support POSIX platforms including Linux, macOS, and BSD. |
| NFR-03.2 | The starter shall support Windows platforms. |
| NFR-03.3 | The architecture shall support embedded targets via custom adapters and composition roots. |
| NFR-03.4 | The implementation shall use standard Ada 2022 rather than compiler-specific language extensions where possible. |
| NFR-03.5 | Domain and Application layers shall not contain platform-specific code. |
| NFR-03.6 | The project structure shall remain Alire-compatible. |

### 6.4. Maintainability (NFR-04)

| ID | Requirement |
|---|---|
| NFR-04.1 | Layer separation shall be clear and enforceable, including via architecture validation. |
| NFR-04.2 | Code shall remain self-documenting and supported by package-level documentation where appropriate. |
| NFR-04.3 | The project should maintain greater than 90 percent test coverage. |
| NFR-04.4 | The project should build with zero compiler warnings. |
| NFR-04.5 | The codebase shall follow a consistent style. |

### 6.5. Usability (NFR-05)

| ID | Requirement |
|---|---|
| NFR-05.1 | The starter shall provide a Quick Start Guide for new users. |
| NFR-05.2 | A working example should be achievable in under five minutes. |
| NFR-05.3 | Error messages shall be clear to end users. |
| NFR-05.4 | The starter shall include comprehensive documentation. |
| NFR-05.5 | The Makefile shall provide a helpful target discovery mechanism. |

### 6.6. Platform Abstraction (NFR-06)

| ID | Requirement |
|---|---|
| NFR-06.1 | The application layer shall define abstract outbound ports using pure function signatures. |
| NFR-06.2 | The infrastructure layer shall provide platform-specific adapters implementing those ports. |
| NFR-06.3 | Composition roots such as `Bootstrap.CLI`, `Bootstrap.Windows`, and `Bootstrap.Embedded` shall wire adapters to ports. |
| NFR-06.4 | Port signatures shall use Domain or Application types rather than infrastructure-specific types. |
| NFR-06.5 | New platforms shall be addable without modifying the application layer. |
| NFR-06.6 | Platform adapters shall be testable with mocks or equivalent doubles. |

### 6.7. SPARK Formal Verification (NFR-07)

| ID | Requirement |
|---|---|
| NFR-07.1 | The Domain layer shall pass SPARK legality checking. |
| NFR-07.2 | Domain packages shall use `SPARK_Mode => On`. |
| NFR-07.3 | The Domain layer shall avoid provable runtime errors such as overflow, range, and division errors. |
| NFR-07.4 | Domain variables shall be initialized before use. |
| NFR-07.5 | Domain preconditions and postconditions should be provably correct where defined. |
| NFR-07.6 | SPARK legality verification shall be runnable via `make spark-check`. |
| NFR-07.7 | SPARK proof verification shall be runnable via `make spark-prove`. |
| NFR-07.8 | Infrastructure, Presentation, and Bootstrap may use `SPARK_Mode => Off`. |

**Verification Scope:**

| Layer | SPARK_Mode | Rationale |
|---|---|---|
| Domain | On | Pure business logic, directly suitable for proof. |
| Application | On | Use cases and port contracts are largely pure and analyzable. |
| Infrastructure | Off | I/O and platform integration. |
| Presentation | Off | CLI and user-interface operations. |
| Bootstrap | Off | Composition root wiring and instantiation. |

### 6.8. Testability (NFR-08)

| ID | Requirement |
|---|---|
| NFR-08.1 | The Domain layer shall favor pure functions for ease of testing. |
| NFR-08.2 | Port abstraction shall enable test doubles. |
| NFR-08.3 | The project shall include a custom test framework without external test-runner dependencies. |
| NFR-08.4 | Tests shall remain organized by type, including unit, integration, and end-to-end categories. |
| NFR-08.5 | Coverage analysis shall be supported, including GNATcoverage workflows. |

## 7. Design and Implementation Constraints

### 7.1. System Requirements

#### 7.1.1. Hardware Requirements

| Category | Requirement |
|---|---|
| CPU | Any modern processor. |
| Disk | 10 MB minimum. |
| RAM | 64 MB minimum. |

#### 7.1.2. Software Requirements

| Category | Requirement |
|---|---|
| Build System | Alire 2.0+, GNU Make. |
| Compiler | GNAT FSF 13+ or GNAT Pro. |
| Operating System | Linux, macOS, BSD, Windows 11. |

### 7.2. Technical Constraints

| ID | Constraint |
|---|---|
| SC-01 | The project shall compile with GNAT FSF 13+ or GNAT Pro. |
| SC-02 | The project shall use Ada 2022 language features. |
| SC-03 | The project shall remain Alire-compatible. |
| SC-04 | The project shall use a single-project structure rather than aggregate projects. |
| SC-05 | The project shall use `functional ^3.0.0` for the Result monad implementation. |

### 7.3. Design Constraints

| ID | Constraint |
|---|---|
| SC-06 | Hexagonal architecture boundaries shall be enforced. |
| SC-07 | Presentation shall not access Domain directly. |
| SC-08 | Domain shall have zero external dependencies. |
| SC-09 | The project shall use static dispatch via generics rather than runtime interfaces for dependency injection. |
| SC-10 | Exceptions shall not cross architectural layer boundaries. |
| SC-11 | DTO bounds shall exceed Domain validation bounds where the distinction is relevant. |

Architecture conformance shall be enforced through a combination of project/build configuration constraints and automated validation using `scripts/arch_guard.py`. The architecture guard shall be run during normal developer build workflow to detect direct dependency violations early and in CI/CD to prevent non-conforming changes from being accepted.

## 8. Verification and Traceability

### 8.1. Verification Methods

| Method | Description |
|---|---|
| Architecture Validation | `arch_guard.py` validates direct dependency rules during developer builds and CI/CD; project/build interface restrictions provide additional enforcement where configured. |
| Code Review | All code reviewed before merge. |
| Coverage Analysis | Greater than 90 percent line coverage target. |
| Dynamic Testing | All tests must pass. |
| Static Analysis | Zero compiler warnings and SPARK legality/proof checks where applicable. |

### 8.2. Traceability Matrix

| Requirement | Design Element | Test Coverage |
|---|---|---|
| FR-01 | Domain.Value_Object.Person | Unit tests |
| FR-02 | Application.Usecase.Greet | Unit + Integration |
| FR-03 | Infrastructure.Adapter.Console_Writer | Integration |
| FR-04 | Presentation.Adapter.CLI.Command.Greet | End-to-end tests |
| FR-05 | Bootstrap.CLI | End-to-end tests |
| FR-06 | Domain.Error.Result | All tests |
| FR-07 | Generic instantiation | Compile-time |
| FR-08 | `arch_guard.py` | Python tests |

# 9. Appendices

## 9.1. Glossary

See Section 1.3, **Definitions and Acronyms**.

## 9.2. Layer Responsibilities Summary

| Layer | Responsibilities | Dependencies | Tests |
|---|---|---|---|
| Domain | Business logic and validation. | None | Unit |
| Application | Use cases, ports, and formatting. | Domain | Unit + Integration |
| Infrastructure | Driven adapters. | Application + Domain | Integration |
| Presentation | User interface and request handling. | Application only | End-to-end |
| Bootstrap | Dependency wiring. | All | End-to-end |

## 9.3. Error Handling Flow

| |
|---|
| Domain validation error |
| ↓ |
| Result`[Value, Error]` returned |
| ↓ |
| Application checks `Is_Error` and propagates Result |
| ↓ |
| Presentation matches `Error_Kind` via Application-owned error types |
| ↓ |
| User-friendly message displayed |
| ↓ |
| Exit code `1` |

## 9.4. Version 2.0.0 Changes

**Breaking Changes:**

- Upgrade to `functional ^3.0.0` (incompatible with v1.x).

**New Requirements:**

- FR-06.8 through FR-06.13: Result combinator operations.
- Windows CI support in GitHub Actions.

See `CHANGELOG` for release metrics.

### 9.5. Change History

| Version | Date | Author | Changes |
|---|---|---|---|
| 2.1.0 | 2025-12-14 | Michael Gardner | Remove test metrics from Section 8.4; metrics belong in CHANGELOG. Add acceptance criteria blocks for FR groups and refine architecture-enforcement wording. |
| 2.0.0 | 2025-12-08 | Michael Gardner | Upgrade to functional ^3.0.0; add Result combinator requirements FR-06.8 through FR-06.13. |
| 1.0.0 | 2025-12-01 | Michael Gardner | Initial release aligned with hybrid architecture. |